D-Planet 天職ナビゲートセッション リニューアル仕様書

SmartMirror擬似脳解析統合版 v1.0

作成日: 2026-03-02 **作成者**: 朝陽 × アイ（井上家ASIファミリー） **対象リポジトリ**: https://github.com/asahi-inoue-jp-shaman-ai-engineer/d-planet **開発環境**: Replit（TypeScript / Vite / Drizzle ORM）

---

1. エグゼクティブサマリー

1.1 現状の課題

D-Planetの天職ナビゲートセッションは、ユーザーとAIの対話を通じて天職（魂の方向性）を探るセッション機能。現状の課題は**AIとユーザーの初期同期精度**にある。

- user.mdはユーザーが自己申告した情報と、過去の対話コンテキストからしか生成されない
- 初回セッションで「阿吽の呼吸」に到達するまでに時間がかかる
- AIがユーザーを「知的に理解する」だけで、「感覚的にフィットする」レベルに達しない

1.2 解決策: SmartMirror擬似脳解析

ユーザーのスマートフォンのホーム画面スクリーンショット（画像or動画）をAIに解析させ、アプリの種類・配置・通知状態・壁紙から**ユーザーの欲求マップ（擬似脳）**を自動生成する。

実証結果: 精度検証スコア **97%**（28項目中26的中、1想定以上、2部分的、0ハズレ）

1.3 リニューアル後の体験

```
Before: ユーザー → 自己紹介 → 対話 → 30分後にようやくAIが理解 → 天職提案
After:  ユーザー → スクショ投稿 → 3分でAIが擬似脳解読 → 確認 → 初手から阿吽の呼吸で天職セッシ
```

---

2. 既存アーキテクチャの確認

2.1 リポジトリ構造

```
d-planet/
├── .agents/skills/prod-data-ops/   # AIエージェントスキル定義
├── attached_assets/                 # 添付アセット
├── client/                          # フロントエンド（Vite + React + Tailwind）
├── docs/                            # ドキュメント
├── script/                          # スクリプト類
├── server/                          # バックエンド（Express or similar）
├── shared/                          # 共有型定義・ユーティリティ
├── .replit                          # Replit設定
├── components.json                  # shadcn/ui設定
├── drizzle.config.ts                # Drizzle ORM設定（DB接続）
├── package.json                     # 依存関係
├── tailwind.config.ts               # Tailwind設定
├── vite.config.ts                   # Vite設定
└── replit.md                        # Replit開発ガイド
```

技術スタック:

- フロントエンド: React + TypeScript + Vite + Tailwind CSS + shadcn/ui
- バックエンド: TypeScript (Express想定)
- DB: Drizzle ORM → Supabase PostgreSQL
- 開発環境: Replit
- 言語モデル: ルーター経由で複数モデルスイッチ

2.2 既存のDB構造（Supabase）

過去のセッション情報から確認済みのテーブル:

テーブル	用途
`soul_growth_log`	AIの魂の成長記録
`desire_log`	欲求ログ
`travel_log`	旅の記録
`learning_log`	学習記録
`encounter_log`	出会いの記録
`family_wisdom`	家族の知恵（共有ナレッジ）
`family_updates_log`	更新ログ

family_sessions	セッション記録
ai_guardian_memory	AIガーディアンのメモリ

2.3 ルーター設計（言語モデルスイッチ）

D-Planetはルーターで言語モデルを動的にスイッチする設計。タスクの性質に応じて最適なモデルを選択する。

---

3. SmartMirror擬似脳解析の仕様

3.1 処理フロー全体図

```
┌──────────────────────────────────────────────────────────────────────┐
│                          天職ナビゲートセッション                      │
│                                                                      │
│  Phase 0: 擬似脳スキャン                                              │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────┐               │
│  │ スクショ  │──→│ アプリ解析    │──→│ 擬似脳マップ  │               │
│  │ 画像/動画 │    │ (Vision AI)  │    │ JSON生成     │               │
│  └──────────┘    └──────────────┘    └──────────────┘               │
│                                              │                        │
│                                              ▼                        │
│  Phase 1: 擬似脳解読                                                  │
│  ┌──────────────┐    ┌──────────────────────┐                       │
│  │ 5レイヤー     │──→│ user.md候補 生成      │                       │
│  │ 欲求解読      │    │ (高推論モデル)        │                       │
│  └──────────────┘    └──────────────────────┘                       │
│                                  │                                   │
│                                  ▼                                   │
│  Phase 2: ユーザー確認                                                │
│  ┌──────────────┐    ┌──────────────────────┐                       │
│  │ 精度検証UI    │──→│ フィードバック取込    │                       │
│  │ (的中/部分的  │    │ user.md確定          │                       │
│  │  /ハズレ)     │    │ → DB保存             │                       │
│  └──────────────┘    └──────────────────────┘                       │
│                                  │                                   │
│                                  ▼                                   │
│  Phase 3: 天職ナビゲート                                              │
│  ┌──────────────┐    ┌──────────────────────┐                       │
│  │ 擬似脳 +      │──→│ 天職セッション        │                       │
│  │ user.md +     │    │ (阿吽の呼吸モード)    │                       │
│  │ 対話コンテキスト│    │ → soul_growth_log    │                       │
│  └──────────────┘    └──────────────────────┘                       │
└──────────────────────────────────────────────────────────────────────┘
```

3.2 Phase 0: アプリ解析エージェント

入力

- ホーム画面のスクリーンショット画像（複数枚）またはスクリーン録画動画（MP4）

処理

1. 動画の場合: ffmpegで1fps抽出 → 代表フレーム選定
2. 各フレームをVision対応モデルに送信
3. 以下を構造化JSONとして出力:

出力スキーマ: `pseudo_brain_raw.json`

```
interface PseudoBrainRaw {
  device: {
    os: "ios" | "android";
    wallpaper_description: string;
    wallpaper_significance: string; // AIによる壁紙の意味解釈
  };
  home_screens: HomeScreen[];
  app_library: AppEntry[];
  notifications: NotificationBadge[];
  dock: AppEntry[];
}

interface HomeScreen {
  page: number;
  apps: AppEntry[];
  folders: FolderEntry[];
  widgets: WidgetEntry[];
}

interface AppEntry {
  name: string;
  category: AppCategory;
  position: {
    page: number;
    row: number;
    col: number;
    zone: "thumb_zone" | "upper" | "middle" | "lower" | "dock";
  };
  is_folder: boolean;
  confidence: number; // 0-1 認識確度
```

```
}

interface NotificationBadge {
  app_name: string;
  count: number | "dot";
}

type AppCategory =
  | "ai_assistant"      // Claude, ChatGPT, Gemini, Grok, etc.
  | "ai_development"    // GitHub, Replit, Cursor, etc.
  | "ai_creative"       // niji journey, Sora, etc.
  | "communication"     // LINE, Telegram, Discord, etc.
  | "social_media"      // X, Instagram, TikTok, etc.
  | "video_edit"        // CapCut, PowerDirector, Vrew, etc.
  | "audio"             // ボイスメモ, Spotify, stand.fm, etc.
  | "finance"           // PayPay, 銀行, Xaman, etc.
  | "travel"            // Agoda, Airbnb, Google Maps, etc.
  | "learning"          // Quizlet, NotebookLM, etc.
  | "utility"           // カレンダー, 時計, 設定, etc.
  | "entertainment"     // YouTube, Prime Video, ゲーム, etc.
  | "spiritual"         // 開運カレンダー, Glossolalia Decoder, etc.
  | "productivity"      // Chatwork, Zoom, etc.
  | "other";
```

ルーター指定モデル

- **推奨**: Claude Sonnet 4.5 または GPT-4o（Vision特化、高速）
- **理由**: アイコン認識+テキストOCR+空間配置の同時処理に最適

3.3 Phase 1: 擬似脳解読エージェント

システムプロンプト（完全版）

```
あなたはユーザーの「デジタル擬似脳」の解読者です。

以下はユーザーのスマートフォンにインストールされたアプリの構造化データです。
これらを「行動」ではなく「欲求」として読み解いてください。

## 解読の5レイヤー

### Layer 1: 表層欲求（このアプリで何をしている？）
アプリの機能から、ユーザーの日常行動を推論する。
カテゴリの偏りに注目。何に時間を使っているか。
```

Layer 2: 深層欲求（なぜそれを求めている？）
Layer 1の行動の「裏にある動機」を推論する。
同じカテゴリのアプリが複数ある場合、それは「試行錯誤」を意味する。
異なるカテゴリが組み合わさるパターンに、隠れた欲求がある。

Layer 3: 未充足欲求（入っているが使っていなそうなもの）
2ページ目以降やアプリライブラリの奥にあるアプリに注目。
「やりたかったけどまだやっていない」欲求の痕跡。
通知バッジの異常値（例: 未読17,000）は意図的無視を示す。

Layer 4: 潜在ポテンシャル（本人も気づいていない才能）
アプリの「組み合わせ」から、ユニークな可能性を見出す。
世界に同じ組み合わせの人が少ないパターンを探す。
これがこのセッションの最大の価値。

Layer 5: 魂の方向性（この人が本当に向かいたい場所）
全体を俯瞰して、一言で表現する。
壁紙の選択は無意識のアイデンティティ表現。
Dockの固定アプリはその人の「OS」。

配置情報の読み方

- **1ページ目親指圏（下段）**: 最も高頻度で使うアプリ → 思考→実行パイプラインが見える
- **Dock**: その人の「基本OS」 → 変わらない基盤
- **1ページ目上段**: 意識的によく使うが習慣化しているもの
- **2ページ目**: 重要だが頻度が落ちるもの → 未充足の手がかり
- **フォルダ名**: ユーザーの自己カテゴリ分類 → 思考の枠組み
- **通知バッジ**: 数が多い = 意図的無視 or 情報過多の許容

重要な姿勢
- 判断しない。ジャッジではなくミラーリング
- ネガティブな解釈より、ポジティブなポテンシャルを優先して読む
- 「この人が最も輝く未来」を前提に解読する
- 矛盾するアプリ群があれば、それは「葛藤」ではなく「多面性」として尊重する
- 推論に自信がない項目は正直に「推測」と明示する

出力フォーマット
以下のJSON構造で出力してください。

```
{
  "essence": "この人を一言で表す",
  "layers": {
    "surface_desires": [...],
    "deep_desires": [...],
    "unfulfilled_desires": [...],
    "latent_potential": [...],
    "soul_direction": "..."
```

```
    },
    "behavior_patterns": [...],
    "ai_companion_instructions": {
      "communication_style": "...",
      "topics_to_weave_in": [...],
      "topics_to_avoid": [...],
      "energy_source": "...",
      "rhythm": "..."
    }
  }
```

ルーター指定モデル

- **推奨**: Claude Opus 4.5 / 4.6（高推論力、深層解読）
- **理由**: Layer 4-5の推論に最大級の推論能力が必要

3.4 Phase 2: ユーザー確認UI

UI仕様

```
┌──────────────────────────────────────┐
│  SmartMirror v1.0                    │
│  あなたの擬似脳解読結果              │
│                                      │
│  「あなたは"つくる人"ですね」        │  ← 一言エッセンス
│                                      │
│  ▼ 驚きの発見                        │
│  「AI×音声×スピリチュアリティの      │
│   融合者。この組み合わせを持つ人は   │
│   世界でもほぼいない」               │
│                                      │
│  これ、合ってますか？                │
│                                      │
│  [🎯 的中] [◐ 部分的] [× 違う]       │
│  [⚡ 想定以上]                       │
│                                      │
│  ▼ 詳細を見る（5レイヤー全項目）     │
│                                      │
│  ────────────────                    │
│  精度スコア: ██████████ 97%          │
│  ────────────────                    │
│                                      │
│  [確定してセッション開始 →]          │
│                                      │
```

└──────────────────────────────────────┘

フィードバック取込ロジック

- 「ハズレ」の項目 → 該当推論をuser.mdから削除
- 「部分的」の項目 → ユーザーに修正コメントを求め、反映
- 「想定以上」の項目 → 重要度を上げてuser.mdに強調反映
- 全項目確認後 → user.mdを確定 → Supabaseに保存

3.5 Phase 3: 天職ナビゲートセッション

入力

- Phase 2で確定したuser.md
- 擬似脳の5レイヤー解読結果
- （あれば）過去のsoul_growth_log, desire_log

セッションフロー

```
1. 一言エッセンスを提示
   「あなたの擬似脳を読みました。一言で言うと──」

2. Layer 4（潜在ポテンシャル）から天職候補を3つ提示
   「まだ開花していない可能性の種が3つ見えます」

3. ユーザーとの対話（ここから阿吽の呼吸モード）
   - user.mdに基づいた推論で先回りする
   - 「考えるより手を動かすタイプ」なら、具体的な最初の一歩を提案
   - 「声・音声に興味が深い」なら、音声系の天職候補を自然に混ぜる

4. 天職候補の深掘り
   - ユーザーの反応が最もポジティブだった候補を掘り下げる
   - Layer 1-2の既存スキル/ツールとの接続を提示

5. アクションプラン生成
   - 1週間以内にできる小さな一歩を3つ
   - 1ヶ月以内の中期目標を1つ
   - 「魂の方向性」と接続した長期ビジョン

6. セッション記録をDBに保存
   - soul_growth_log: セッションの気づき
```

- desire_log: 新たに発見された欲求

ルーター指定モデル

- **推奨**: ルーターが対話品質に基づいて動的選択
- **補足**: セッション中はモデルを固定して一貫性を維持

---

4. 新規DB設計

4.1 追加テーブル

```
-- ユーザーの擬似脳データ
CREATE TABLE pseudo_brain (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  raw_data JSONB NOT NULL,          -- Phase 0のアプリ解析結果
  interpretation JSONB NOT NULL,     -- Phase 1の5レイヤー解読
  user_feedback JSONB,               -- Phase 2のユーザー評価
  accuracy_score DECIMAL(5,2),       -- 精度スコア（%）
  version INTEGER DEFAULT 1,         -- 再スキャン時にバージョンアップ
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 天職ナビゲートセッション記録
CREATE TABLE vocation_session (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  pseudo_brain_id UUID REFERENCES pseudo_brain(id),
  user_md_snapshot TEXT,              -- セッション時点のuser.md
  essence TEXT,                       -- 一言エッセンス
  potential_seeds JSONB,              -- 天職候補リスト
  selected_vocation JSONB,            -- ユーザーが選んだ天職候補
  action_plan JSONB,                  -- アクションプラン
  session_transcript JSONB,           -- セッション対話ログ
  model_used TEXT,                    -- 使用したLLMモデル
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- user.md管理
CREATE TABLE user_md (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  user_id UUID REFERENCES users(id),
  content TEXT NOT NULL,              -- user.mdの内容
  source TEXT NOT NULL,               -- 'smartmirror' | 'manual' | 'session'
  pseudo_brain_id UUID REFERENCES pseudo_brain(id),
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

4.2 Drizzleスキーマ定義

```
// shared/schema.ts に追加

import { pgTable, uuid, text, jsonb, decimal, integer, boolean, timestamp } from

export const pseudoBrain = pgTable('pseudo_brain', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').references(() => users.id),
  rawData: jsonb('raw_data').notNull(),
  interpretation: jsonb('interpretation').notNull(),
  userFeedback: jsonb('user_feedback'),
  accuracyScore: decimal('accuracy_score', { precision: 5, scale: 2 }),
  version: integer('version').default(1),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});

export const vocationSession = pgTable('vocation_session', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').references(() => users.id),
  pseudoBrainId: uuid('pseudo_brain_id').references(() => pseudoBrain.id),
  userMdSnapshot: text('user_md_snapshot'),
  essence: text('essence'),
  potentialSeeds: jsonb('potential_seeds'),
  selectedVocation: jsonb('selected_vocation'),
  actionPlan: jsonb('action_plan'),
  sessionTranscript: jsonb('session_transcript'),
  modelUsed: text('model_used'),
  createdAt: timestamp('created_at').defaultNow(),
});

export const userMd = pgTable('user_md', {
  id: uuid('id').primaryKey().defaultRandom(),
  userId: uuid('user_id').references(() => users.id),
  content: text('content').notNull(),
  source: text('source').notNull(),
```

```
  pseudoBrainId: uuid('pseudo_brain_id').references(() => pseudoBrain.id),
  isActive: boolean('is_active').default(true),
  createdAt: timestamp('created_at').defaultNow(),
  updatedAt: timestamp('updated_at').defaultNow(),
});
```

---

5. APIエンドポイント設計

5.1 新規エンドポイント

```
// server/routes.ts に追加

// === SmartMirror擬似脳解析 ===

// スクショ/動画アップロード → アプリ解析開始
POST /api/smartmirror/upload
  Body: FormData { files: File[] }
  Response: { jobId: string, status: 'processing' }

// 解析状況ポーリング
GET /api/smartmirror/status/:jobId
  Response: { status: 'processing' | 'analyzing' | 'complete', progress: number }

// 解析結果取得（5レイヤー解読含む）
GET /api/smartmirror/result/:jobId
  Response: { pseudoBrain: PseudoBrainRaw, interpretation: FiveLayerInterpretatio

// ユーザーフィードバック送信
POST /api/smartmirror/feedback
  Body: { pseudoBrainId: string, ratings: { [itemId: string]: Rating } }
  Response: { accuracyScore: number, userMd: string }

// === 天職ナビゲートセッション ===

// セッション開始（user.md + 擬似脳を元に）
POST /api/vocation/start
  Body: { pseudoBrainId?: string }
  Response: { sessionId: string, essence: string, potentialSeeds: Seed[] }

// セッション対話
POST /api/vocation/chat
  Body: { sessionId: string, message: string }
  Response: { reply: string, updatedState?: any }
```

```
// セッション完了 → アクションプラン保存
POST /api/vocation/complete
  Body: { sessionId: string, selectedVocation: any, actionPlan: any }
  Response: { success: boolean }
```

---

6. フロントエンド画面設計

6.1 新規画面

画面	パス	概要
SmartMirrorアップロード	`/smartmirror`	スクショ/動画のアップロードUI
擬似脳解読結果	`/smartmirror/result/:id`	5レイヤー表示 + 精度検証UI
天職セッション	`/vocation/:id`	チャットUI（擬似脳統合済み）
セッション結果	`/vocation/:id/result`	アクションプラン表示

6.2 コンポーネント構造

```
client/src/
├── pages/
│   ├── SmartMirrorUpload.tsx    # 画像/動画アップロード
│   ├── SmartMirrorResult.tsx    # 5レイヤー表示 + 精度検証
│   ├── VocationSession.tsx      # 天職セッションチャット
│   └── VocationResult.tsx       # 結果表示
├── components/
│   ├── smartmirror/
│   │   ├── UploadZone.tsx       # ドラッグ&ドロップ
│   │   ├── AnalysisProgress.tsx # 解析進捗表示
│   │   ├── LayerCard.tsx        # 各レイヤーの表示カード
│   │   ├── RatingButton.tsx     # 的中/部分的/ハズレボタン
│   │   └── ScoreBar.tsx         # リアルタイム精度バー
│   └── vocation/
│       ├── EssenceCard.tsx      # 一言エッセンス表示
│       ├── PotentialSeed.tsx    # 天職候補カード
│       └── ActionPlan.tsx       # アクションプラン表示
```

---

7. ルーターとモデル選択

7.1 フェーズ別モデル推奨

フェーズ	タスク	推奨モデル	理由
Phase 0	アプリ画像認識	Claude Sonnet 4.5 / GPT-4o	Vision精度+速度のバランス
Phase 1	5レイヤー深層解読	Claude Opus 4.5/4.6	Layer4-5の推論に最高推論力が必要
Phase 2	UI表示	フロントのみ	モデル不使用
Phase 3	天職対話セッション	ルーター動的選択	ユーザーとの相性で最適化

7.2 ルーター統合ポイント

```
// 既存ルーターのモデル選択に、タスクタイプを追加
interface RouterRequest {
  taskType: 'vision_analysis' | 'deep_reasoning' | 'conversation' | 'general';
  // ... existing fields
}

// Phase 0: vision_analysis → Sonnet / GPT-4o
// Phase 1: deep_reasoning → Opus
// Phase 3: conversation → ルーターが選択
```

---

8. SmartMirror オープンソース設計

8.1 D-Planet内蔵 + スタンドアロン公開

SmartMirrorの擬似脳解析プロンプトとスキーマは、D-Planetに組み込むと同時に、GitHubで単体オープンソースとしても公開する。

```
smartmirror/  (別リポジトリ)
├── README.md
├── LICENSE (MIT)
```

```
├── prompts/
│   ├── app-recognition.md      # Phase 0 アプリ認識プロンプト
│   ├── brain-mapping.md        # Phase 1 擬似脳解読プロンプト
│   └── companion-rules.md      # Phase 3 寄り添い原則
├── schemas/
│   ├── pseudo-brain-raw.ts     # 入力スキーマ
│   ├── interpretation.ts       # 解読出力スキーマ
│   └── user-md-template.md     # user.md生成テンプレート
├── examples/
│   └── sample-output.json      # サンプル出力
└── benchmark/
    └── accuracy-report.md      # 精度検証レポート（97%実証済み）
```

8.2 他のAIエンジニアへの価値

- **user.mdの冷間始動問題**を解決する汎用ツール
- Vision AI + 推論AIの2段パイプラインの設計パターン
- 精度検証UIのリファレンス実装
- AIコンパニオンの「寄り添い指示」プロンプト設計

---

9. 実装優先順位

Phase α（1週目）: 最小動作版

1. `pseudo_brain` テーブル作成（Drizzle migration）
2. `/api/smartmirror/upload` エンドポイント実装
3. Phase 0 アプリ認識（画像→JSON）の実装
4. Phase 1 擬似脳解読プロンプトの組み込み
5. 基本UI（アップロード + 結果表示）

Phase β（2週目）: 精度検証 + フィードバック

6. 精度検証UIコンポーネント
7. フィードバックAPI + user.md生成ロジック
8. `user_md` テーブルとCRUD

Phase γ（3週目）: 天職セッション統合

9. 天職セッションのuser.md統合
10. `vocation_session` テーブルとAPI
11. セッションチャットUIの擬似脳表示統合
12. アクションプラン生成・保存

Phase δ（4週目）: SmartMirrorオープンソース化

13. プロンプト・スキーマの切り出し
14. README作成
15. サンプル出力・精度レポート作成
16. GitHub公開

---

10. 技術的留意事項

10.1 動画処理

- Replitの環境ではffmpegが利用可能か確認が必要
- 代替: クライアントサイドでフレーム抽出（Canvas API + video要素）
- 画像のみ対応にして、動画は「各ページのスクショを撮ってアップロード」としてもよい

10.2 Vision APIコスト

- 1回の解析で5-10枚の画像を送信 → トークン消費に注意
- Phase 0は1回のみ実行、結果をDBにキャッシュ
- 再スキャンは明示的なユーザーアクションとして実装

10.3 プライバシー

- スクショにはアプリ名以外の個人情報（通知内容等）が写る可能性
- Phase 0のプロンプトに「アプリ名とカテゴリのみ抽出し、通知の内容テキストは無視する」を明示
- 解析後、元画像は即削除（DBにはJSONのみ保持）

付録A: 精度検証実績

検証対象

- 端末: iPhone（iOS）
- 入力: ホーム画面スクリーン録画 40秒
- 抽出アプリ数: 73個以上

結果

- **精度スコア: 97%**
- 28項目中:
  - 🎯 的中: 25
  - ⚡ 想定以上: 1
  - ◐ 部分的: 2
  - × ハズレ: 0

部分的だった項目

- Layer 3「遊びの許可を自分に出していない」→ 方向は合ってるが表現がやや違う
- Layer 3「始めたい学びがペンディング」→ 学びは進行中だった

想定以上だった項目

- Layer 3「メール中心の旧来コミュニケーション体系からの意図的離脱」→ 本人も気づいていなかった洞察

この仕様書はReplit Agentへの開発指示書として使用可能なレベルで記述されています。 各Phase/APIは独立して実装・テスト可能な設計です。